

시스템 제어실험 미로탈출 챌린지 보고서

성명	채현기	학번	20215060
분반	02분반	조	1조
제목	채현기-20215060-1조-챌린지		

1. 탈출 전략 설명 (코드 포함)

1번 코드

```
#-*- coding: utf-8 -*-
```

```
import time
```

```
import numpy as np
```

```
def fun_cal(bbox: list) -> tuple:
```

```
    """
```

```
    게인 스케줄링이 적용된 비례 제어 함수.
```

```
    목표와의 거리에 따라 제어 게인(민감도)을 동적으로 변경합니다.
```

```
    """
```

```
    x, y, w, h = bbox
```

```
    # 게인 스케줄링: 측정한 데이터를 기반으로 기준값 설정
```

```
    if w < 250: # 멀리 있을 때 (1m 이상)
```

```
        KP_vy = 0.1
```

```
        KP_vz = 0.175
```

```
    else: # 가까이 있을 때 (1m 이내)
```

```
KP_vy = 0.1
```

```
KP_vz = 0.175
```

```
y_offset = -70
```

```
# 추천 파라미터 값 적용
```

```
vx = 15
```

```
vy = -KP_vy * x
```

```
vz = KP_vz * y
```

```
return vx, vy, vz
```

```
def compute(label: int, bbox: list, pitch:int, roll:int) -> list:
```

```
    """
```

```
    드론의 전체 동작을 제어하는 메인 함수
```

```
    """
```

```
# 1. 제어 변수 초기화
```

```
land_flag = False
```

```
vx, vy, vz, yaw_rate = 0, 0, 0, 0
```

```
x, y, w, h = bbox
```

```
# 2. 상태 변수 초기화 (최초 1회만 실행)
```

```
if not hasattr(compute, "drone_state"):
```

```
    compute.drone_state = 'TRACKING' # 'TRACKING' 또는 'SEARCHING'
```

```
    compute.search_yaw_rate = 0      # 회전 방향 및 속도 저장
```

```
    compute.new_label_detected_time = 0 # 새로운 라벨이 마지막으로 보인 시간
```

```
    compute.last_known_label = 0     # 마지막으로 확정된 라벨
```

3. 목표 크기 설정

target_w, target_h = 400, 280

4. 메인 제어 로직

'탐색' 상태일 때의 로직 (수정된 부분)

if compute.drone_state == 'SEARCHING':

새로운 목표물(label!=0)이 '보이기 시작'했다면x

if label != 0:

처음 보는 라벨이면, 탐지 시간을 기록 시작

if compute.last_known_label != label:

compute.last_known_label = label

compute.new_label_detected_time = time.time()

0.5초 이상 꾸준히 보였는지 확인하여 신뢰성 확보

if time.time() - compute.new_label_detected_time > 0.5:

print("New target confirmed! Switching to TRACKING mode.")

compute.drone_state = 'TRACKING'

else:

아직 확신할 수 없으므로(0.5초 미만) 계속 회전

yaw_rate = compute.search_yaw_rate

계속 아무것도 보이지 않는다면 (label == 0)

else:

탐지 시간 및 라벨 관련 변수 초기화 후 계속 회전

```

compute.new_label_detected_time = 0

compute.last_known_label = 0

print("Searching for the next target...")

vx, vy, vz = 0, 0, 0

yaw_rate = compute.search_yaw_rate


# ## '추적' 상태일 때의 로직 ##

if compute.drone_state == 'TRACKING':

    # 화살표가 보이면

    if label == 2 or label == 3:

        if w < target_w or h < target_h:

            vx, vy, vz = fun_cal(bbox)

        else:

            # 목표 도달! '탐색' 상태로 전환하고 회전 방향 결정

            print("Target reached. Switching to SEARCHING mode.")

            compute.drone_state = 'SEARCHING'

            compute.search_yaw_rate = 35 if label == 2 else -35

            yaw_rate = compute.search_yaw_rate # 첫 회전 시작


# 착륙 지점이 보이면

elif label == 1:

    if w < target_w or h < target_h:

        vx, vy, vz = fun_cal(bbox)

    else:

        vx, vy, vz, yaw_rate = 0, 0, 0, 0

```

```
land_flag = True
```

```
# 5. 최종 명령 생성 및 반환
```

```
ctrl_law = [vx, vy, vz, yaw_rate, land_flag]
```

```
ctrl_law_int = [int(v) for v in ctrl_law]
```

```
ctrl_law_int_clipped = [max(min(v, 100), -100) for v in ctrl_law_int]
```

```
return ctrl_law_int_clipped
```

2번 코드

```
#-*- coding: utf-8 -*-
```

```
import time
```

```
import numpy as np
```

```
def fun_cal(bbox: list) -> tuple:
```

```
    """
```

```
    게인 스케줄링이 적용된 비례 제어 함수.
```

```
    목표와의 거리에 따라 제어 게인(민감도)을 동적으로 변경합니다.
```

```
    """
```

```
    x, y, w, h = bbox
```

```
    # 게인 스케줄링: 측정한 데이터를 기반으로 기준값 설정
```

```
    if w < 250: # 멀리 있을 때 (1m 이상)
```

```
        KP_vy = 0.1
```

```
        KP_vz = 0.175
```

```
else: # 가까이 있을 때 (1m 이내)
```

```
    KP_vy = 0.1
```

```
    KP_vz = 0.175
```

```
y_offset = -70
```

```
# 추천 파라미터 값 적용
```

```
vx = 25
```

```
vy = -KP_vy * x
```

```
vz = KP_vz * y
```

```
return vx, vy, vz
```

```
def compute(label: int, bbox: list, pitch:int, roll:int) -> list:
```

```
    """
```

```
    드론의 전체 동작을 제어하는 메인 함수
```

```
    """
```

```
# 1. 제어 변수 초기화
```

```
land_flag = False
```

```
vx, vy, vz, yaw_rate = 0, 0, 0, 0
```

```
x, y, w, h = bbox
```

```
# 2. 상태 변수 초기화 (최초 1회만 실행)
```

```
if not hasattr(compute, "drone_state"):
```

```
    compute.drone_state = 'TRACKING' # 'TRACKING' 또는 'SEARCHING'
```

```
    compute.search_yaw_rate = 0 # 회전 방향 및 속도 저장
```

```
    compute.new_label_detected_time = 0 # 새로운 라벨이 마지막으로 보인 시간
```

```
compute.last_known_label = 0          # 마지막으로 확정된 라벨
```

```
# 3. 목표 크기 설정
```

```
target_w, target_h = 400, 280
```

```
# 4. 메인 제어 로직
```

```
# ## '탐색' 상태일 때의 로직 (수정된 부분) ##
```

```
if compute.drone_state == 'SEARCHING':
```

```
    # 새로운 목표물(label!=0)이 '보이기 시작'했다면x
```

```
    if label != 0:
```

```
        # 처음 보는 라벨이면, 탐지 시간을 기록 시작
```

```
        if compute.last_known_label != label:
```

```
            compute.last_known_label = label
```

```
            compute.new_label_detected_time = time.time()
```

```
    # 0.5초 이상 꾸준히 보였는지 확인하여 신뢰성 확보
```

```
    if time.time() - compute.new_label_detected_time > 0.5:
```

```
        print("New target confirmed! Switching to TRACKING mode.")
```

```
        compute.drone_state = 'TRACKING'
```

```
    else:
```

```
        # 아직 확신할 수 없으므로(0.5초 미만) 계속 회전
```

```
        yaw_rate = compute.search_yaw_rate
```

```
# 계속 아무것도 보이지 않는다면 (label == 0)
```

```
else:
```

```
# 탐지 시간 및 라벨 관련 변수 초기화 후 계속 회전
```

```
compute.new_label_detected_time = 0
```

```
compute.last_known_label = 0
```

```
print("Searching for the next target...")
```

```
vx, vy, vz = 0, 0, 0
```

```
yaw_rate = compute.search_yaw_rate
```

```
# ## '추적' 상태일 때의 로직 ##
```

```
if compute.drone_state == 'TRACKING':
```

```
    # 화살표가 보이면
```

```
    if label == 2 or label == 3:
```

```
        if w < target_w or h < target_h:
```

```
            vx, vy, vz = fun_cal(bbox)
```

```
        else:
```

```
            # 목표 도달! '탐색' 상태로 전환하고 회전 방향 결정
```

```
            print("Target reached. Switching to SEARCHING mode.")
```

```
            compute.drone_state = 'SEARCHING'
```

```
            compute.search_yaw_rate = 35 if label == 2 else -35
```

```
            yaw_rate = compute.search_yaw_rate # 첫 회전 시작
```

```
# 착륙 지점이 보이면
```

```
elif label == 1:
```

```
    if w < target_w or h < target_h:
```

```
        vx, vy, vz = fun_cal(bbox)
```

```
    else:
```



```
vx, vy, vz, yaw_rate = 0, 0, 0, 0
```

```
land_flag = True
```

5. 최종 명령 생성 및 반환

```
ctrl_law = [vx, vy, vz, yaw_rate, land_flag]
```

```
ctrl_law_int = [int(v) for v in ctrl_law]
```

```
ctrl_law_int_clipped = [max(min(v, 100), -100) for v in ctrl_law_int]
```

```
return ctrl_law_int_clipped
```

2. 챌린지 고찰

1번 문항에 코드만 입력하고, 자세한 탈출 전략과 그에 따른 결과 고찰은 2번 문항에서 서술하겠습니다. 코드에 탈출 전략까지 겹쳐 서술하게 되면, 조교님들의 가독성이 매우 떨어질 것 같다고 판단하였습니다. 양해해주시면 감사하겠습니다.

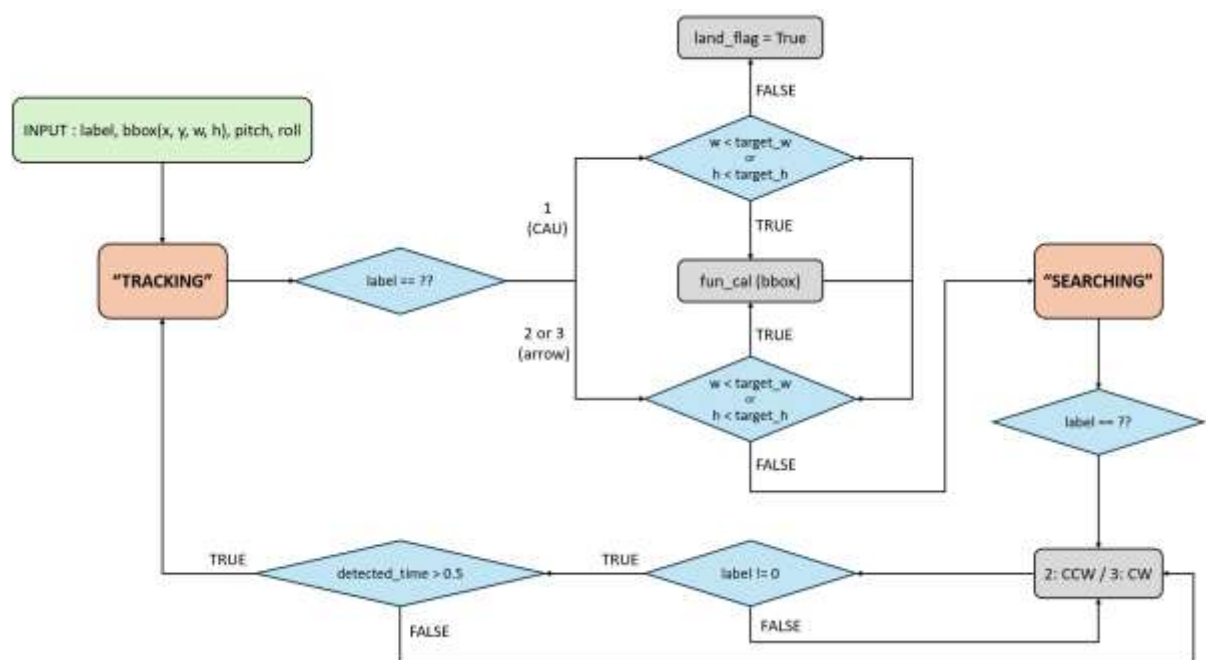
1) 개요

기본 코드의 로직은 다음과 같습니다.

직진함과 동시에 대상을 트래킹하며, 목표크기에 트래킹이 도달하면 곧바로 서칭으로 전환하여 인식한 화살표 방향에 따라 회전 속도와 방향을 설정하였습니다. 다음 화살표를 인식할 때까지 계속 회전하고, 새 화살표가 0.5초 연속 노출되면 해당 화살표를 목표로 트래킹하여 다시 직진하도록 코딩하였습니다. 최종적으로 중앙대학교 마크를 보면 챌린지가 종료되며 착륙까지 하도록 코드를 완성하였습니다.

트래킹을 위한 바운딩박스의 크기값은 $target_w=400$, $target_h=280$ 으로 설정하였습니다. 사전에 주어졌던 2차레의 모의구동을 통해 실험적으로 확인한 최적의 값입니다. 전진구동($v_x=15$)을 기본으로 설정하되, $v_y=-0.1*x$ 와 $v_z=0.175*y$ 를 평면에 연동 보정하여 이미지의 중심에 드론이 최대한 정확히 접근할 수 있도록 보조툴을 마련하였습니다.

더 자세한 설명을 위하여 flow chart를 첨부하였습니다.



2) 전략

1번 코드와 2번 코드 모두 안정성을 최우선으로 짠 코드였습니다. 미로 자체가 매우 크거나 이동 거리가 길다고 판단하지 않아서, 속도를 빠르게 설정하는 대신 인식과 회전을 안정적으로 하는 것이 더 좋을 것 같다는 의도였습니다. 또한 미로의 구조를 미리 알 수 없었기에, 드론의 구동을 최대한 안정적으로 설정해야 한다고 생각하였습니다.

이 과정에서 전진속도(vx)를 낮게 설정하고 상태 전환 속도도 보수적으로 코딩하여 통제 불가능한 상황을 최소화하고자 하였습니다. 또한 1번 코드가 성공적으로 잘 가동될 경우를 가정하여, 1번 코드보다 전진속도(vx)를 빠르게 설정한 코드를 2번 코드로 두었습니다.

저희는 이러한 기조를 검출 시간에도 반영하였습니다. 타 팀의 설정 시간값을 알 수는 없지만, 최대한 안전에 초점을 맞추기 위하여 0.5초라는 지속 검출 시간을 설정하여 드론이 잘못된 인식을 하는 일을 최소화하고자 하였습니다.

3)결과&고찰

저희의 주요 전략에 맞게, 두 차례의 시도에서 모두 완주에 성공하였습니다. 시도 1에서는 전진 속도(vx)를 15로 설정하여 주행 시간이 4분 11초로 길게 소요되었지만, 안정적으로 전 과정을 마칠 수 있었습니다.

시도2에서는 vx를 25로 상향하여 주행 시간이 대폭 단축되었으며, 이를 통해 "안전성을 유지하면서 속도를 높인다"는 전략적 코드 배분이 유효했음을 확인할 수 있었습니다. 결과적으로 안정성이라는 저희의 최초 우선 목적에는 충족하는 결과를 얻었습니다.

다만 시도1에서 시간이 길어진 원인으로는, 드론이 멀리 있는 화살표를 즉각적으로 인식하지 못했기 때문이라고 분석하였습니다. 이는 인식과정에서의 노이즈와 드론 영상 처리 지연 등이 원인일 수 있다고 생각합니다.

또한 코드에서 설정한 목표 크기(target_w, target_h) 조건에 따라 가까이 접근해야만 화살표가 유효하게 판정되었기에, 화살표가 멀리 있을 때는 바운딩박스 크기가 작아 인식 신뢰도 확보에 시간이 걸리고, 그 과정에서 탐색 모드가 불필요하게 길어져 전체 소요 시간이 증가하였을 것이라고 추론하였습니다.

저희 팀만 모든 팀 중, 2회 시도에서 2번 모두 완주에 성공하였지만 저희보다 훨씬 빠른 완주 시간을 기록한 팀도 있었습니다. 이는 안정성보다 속도를 우선한 결과라 할 수 있었으며, 저희와는 뚜렷한 전략적 대비를 보였습니다. 허나, 각 팀의 1,2회차 중 최고 기록을 기준으로 심사가 진행되었기에 결과적으로 저희 팀의 작전 계획과 전략에 아쉬운 점이 있었다고 생각하였습니다.

4주 간의 실험을 통해 얻은 가장 큰 교훈은 안전성과 속도의 균형이라고 생각합니다. 실제 전쟁, 탐사, 구조 등 다양한 상황에서 드론이 투입될 경우, 단순히 정확하게 목표를 찾는 것뿐만 아니라 시간 효율성 또한 임무 성공의 핵심 요인라고 판단됩니다. 따라서 저희의 접근은 안정성 측면에서는 성공적이었으나, 소요 시간을 고려했을 때 다소 아쉬움이 있었습니다.

향후 진행할 실험에서는 안전성을 확보하면서도 속도 최적화에도 주안점을 둘 필요가 있다고 고찰하였습니다.

끝으로, 4주 동안 실험이 원활히 진행될 수 있도록 지도와 수고를 아끼지 않으신 교수님과 조교님께 깊은 감사의 말씀을 드리며 본 보고서를 마치겠습니다.